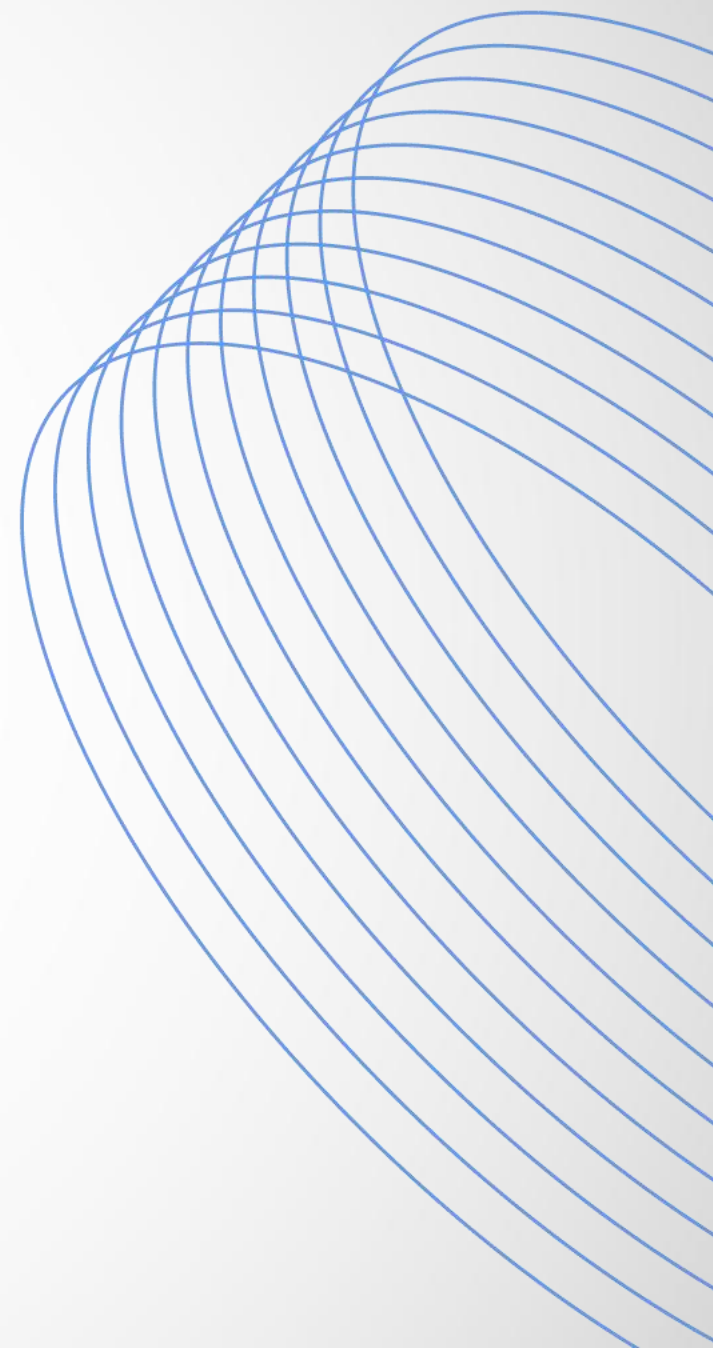


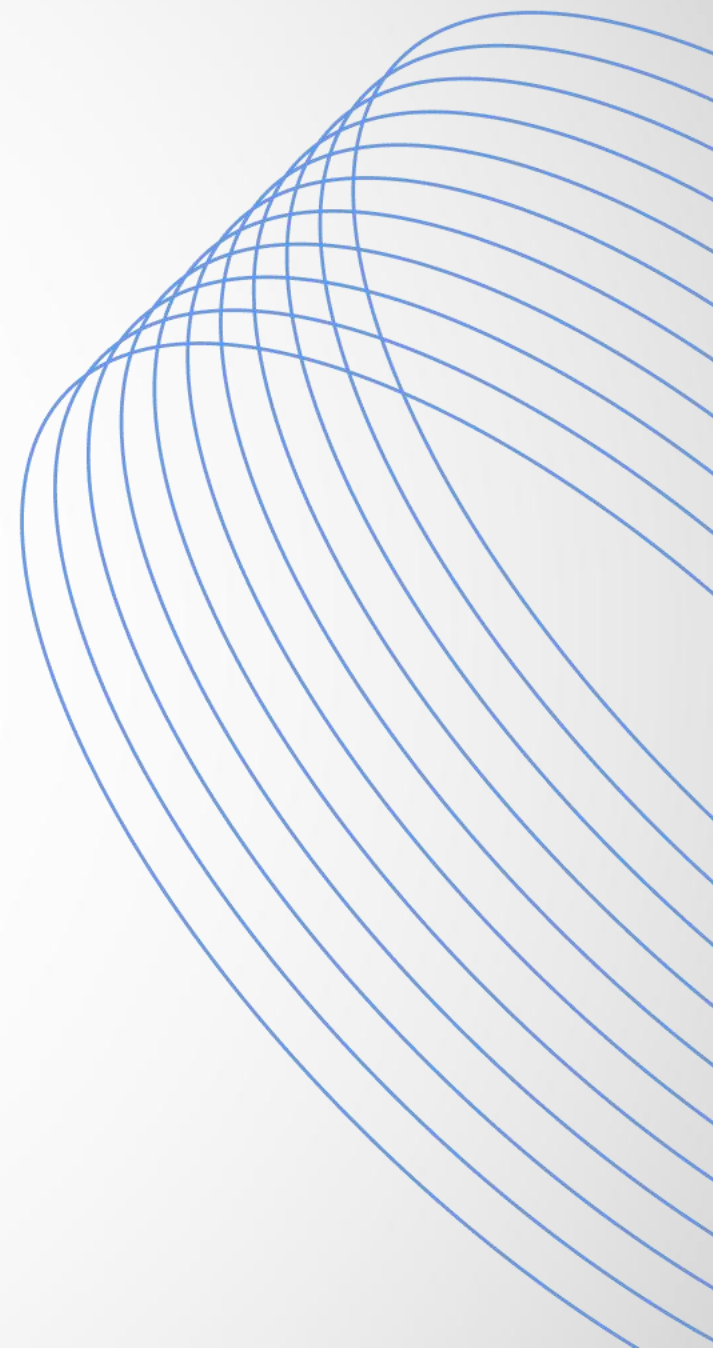
Сжатие изображений нейросетями

Выполнил: Батюченко Дмитрий



Содержание

- Введение
- Коротко о проблемах при работе
- Архитектура нейронной сети
- Что делает каждый файл?
- Демонстрация работы программ
- Итоги



Введение

Цель проекта:

Обучить автоэнкодер на датасете лиц (возможные варианты фрукты, животные, машины), реализовать сжатие изображений с помощью нейросети и сравнить результат по размеру и качеству.

Задачи:

- Найти и подготовить качественный датасет с изображениями лиц
- Реализовать обучение автоэнкодера
- Разработать функции сжатия и восстановления изображений
- Реализовать метрики оценки качества (PSNR, визуальное сравнение) и степени сжатия
- Сравнить результаты с традиционными методами сжатия.



Проблемы с которыми пришлось столкнуться

Потеря качества при сжатии

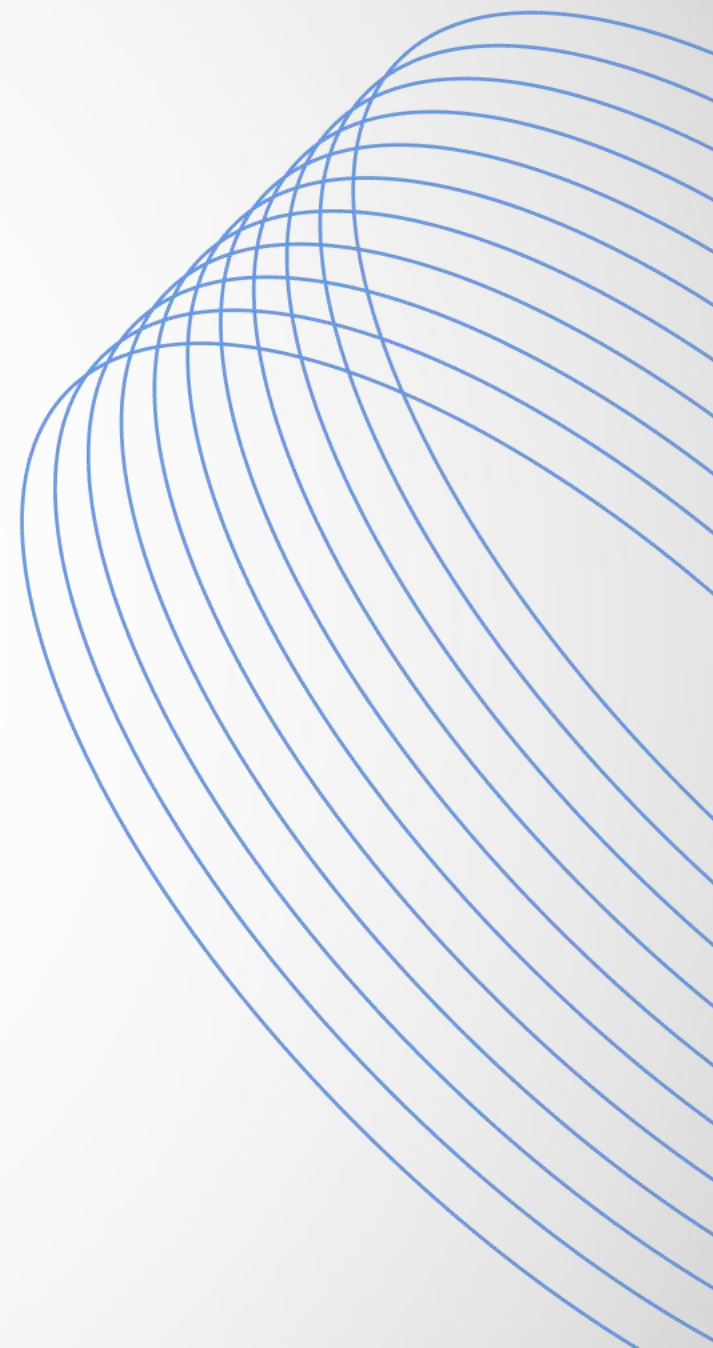
На начальных этапах модель выдавала размытые изображения. Для улучшения PSNR потребовалось улучшить размер латентности пространства.

Время обучения

Поскольку изначально работа выполнялась на CPU время обучения было долгим, но с переходом на GPU время уменьшилось с 2 часов до 15 минут.

Поиск датасета

Большинство датасетов содержало изображения низкое качество неудобный формат, что потребовало времени на подходящий вариант



Архитектура нейронной сети

Архитектура содержит в себе:

Encoder:

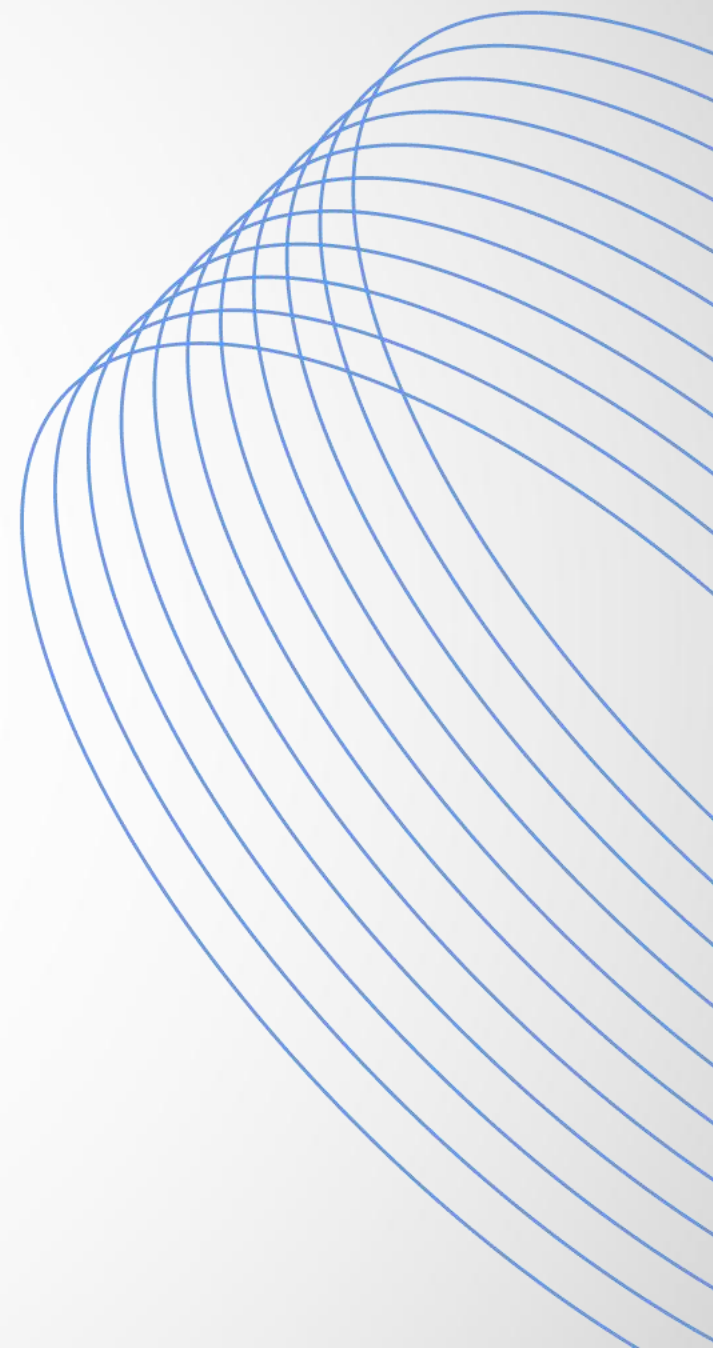
- 4 свёрточных слоя с понижением размерности (stride=2)
- Размерность входа: 256 x 256 x 3
- Латентное пространство: 16 x 16 x 16
- Активации: ReLU + Sigmoid на выходе

Decoder:

- 4 транспонированных свёрточных слоя
- Постепенное увеличение размерности обратно до 256x256
- Активации: ReLU + Sigmoid на выходе

Особенности:

- Латентное представление сжимается приблизительно в 256 раз по сравнению с исходным изображением
- Обучение с функцией потерь MSE



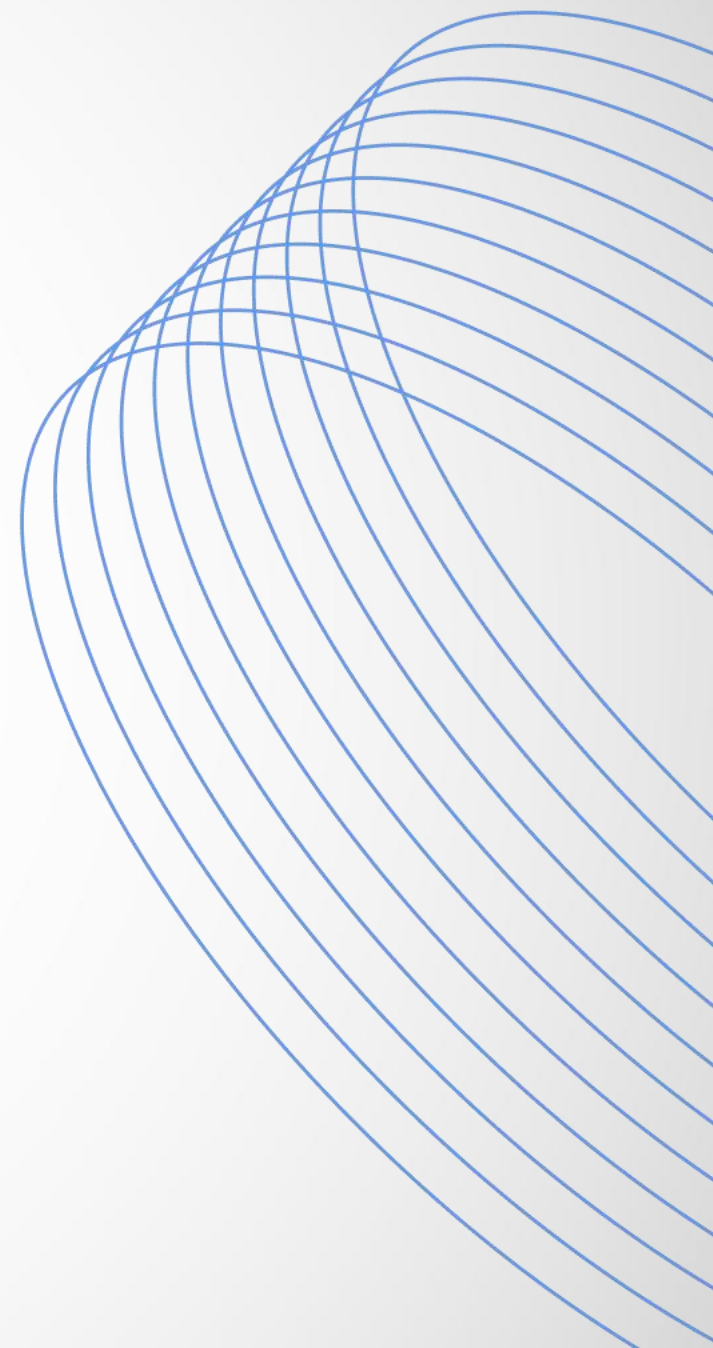
Коротко о файлах

model.py - содержит в себе функцию сжатия и расжатия изображения

train.py - обучает модель на фотографиях

demo.py - показывает начальный и конечный результат изображения (до сжатия и после), а так же при выводится начальный и конечный размер изображения.

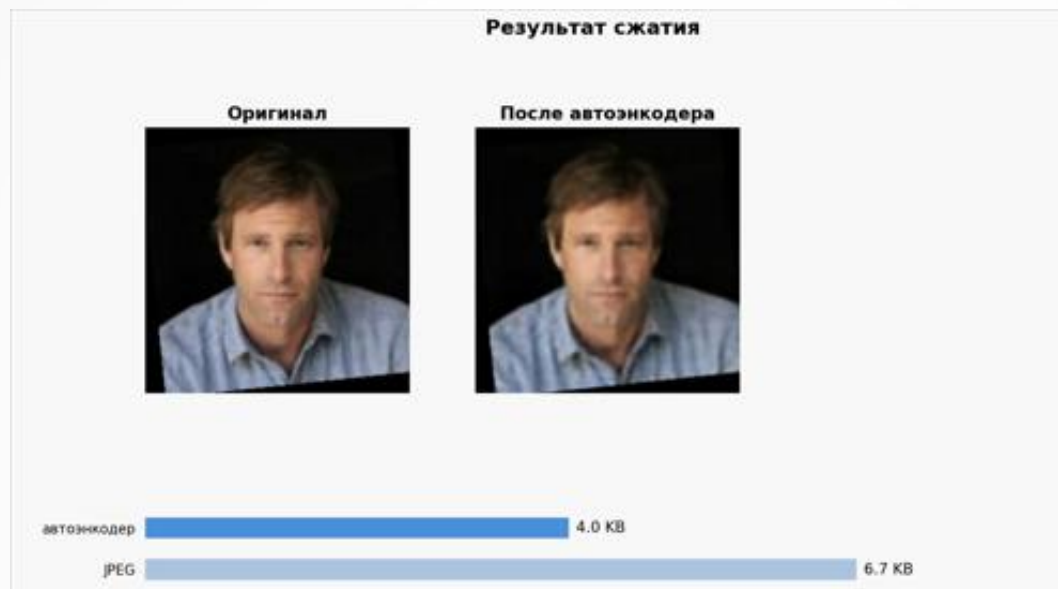
compress.py - выводит в консоль размер оригинального и сжатого изображения во сколько раз сжато изображение, а также PSNR (пиковое отношение сигнала к шуму чем больше, тем лучше).



Демонстрация работы программ

```
Эпоха 1/30 | Loss: 0.0188
Эпоха 2/30 | Loss: 0.0045
Эпоха 3/30 | Loss: 0.0030
Эпоха 4/30 | Loss: 0.0024
Эпоха 5/30 | Loss: 0.0021
Эпоха 6/30 | Loss: 0.0017
Эпоха 7/30 | Loss: 0.0015
Эпоха 8/30 | Loss: 0.0015
Эпоха 9/30 | Loss: 0.0013
Эпоха 10/30 | Loss: 0.0013
Эпоха 11/30 | Loss: 0.0012
Эпоха 12/30 | Loss: 0.0011
Эпоха 13/30 | Loss: 0.0011
Эпоха 14/30 | Loss: 0.0011
Эпоха 15/30 | Loss: 0.0011
Эпоха 16/30 | Loss: 0.0010
Эпоха 17/30 | Loss: 0.0010
Эпоха 18/30 | Loss: 0.0009
Эпоха 19/30 | Loss: 0.0010
Эпоха 20/30 | Loss: 0.0009
Эпоха 21/30 | Loss: 0.0009
Эпоха 22/30 | Loss: 0.0009
Эпоха 23/30 | Loss: 0.0009
Эпоха 24/30 | Loss: 0.0009
Эпоха 25/30 | Loss: 0.0009
Эпоха 26/30 | Loss: 0.0008
Эпоха 27/30 | Loss: 0.0008
Эпоха 28/30 | Loss: 0.0008
Эпоха 29/30 | Loss: 0.0008
Эпоха 30/30 | Loss: 0.0008
```

train.py



demo.py

```
JPEG: 6.7 KB
автоэнкодер: 4.0 KB
Сжатие vs PNG: 1.7x
PSNR: 34.4 dB
```

compress.py

Подведение итогов

Результат:

Реализован автоэнкодер для сжатия изображений лиц

Модель обучена на датасете из 13 000 изображений

Разработаны модули сжатия, восстановления и оценки качества

Достигнуто значительное сжатие изображений при сохранении визуально приемлемого качества.

